SOFTWARE QUALITY ENGINEERING

Course Code: SEC321

Project Title:

ONLINE BANKING SYSTEM

Submitted by:

Mudassir Ghazi 41015

Shahab ul din Khurram 41614

Muhammad Abubakar Farooq 41477

Shamas Ahmad Abbasi 41151

Abdullah Mohsin 41207

Submitted to:

Ms. Madiha Jaffer

Department:

Department of Software Engineering

Date:16/06/2026

Table of Contents

SECTION 1 SYSTEM ANALYSIS AND QUALITY REQUIREMENTS 3

1.1 Functional Requirements 4

1.2 Non-Functional Requirements 5

1.3 Quality Attributes 7

SECTION 2 TEST CASE DESIGN 8

2.1 Equivalence Partitioning 9

2.2 Login Module Test Cases 9

2.3 Boundary Value Analysis 13

2.4 State Transition Testing 16

SECTION 3 WHITE BOX TESTING APPLICATIONS 18

3.1 Control Flow Analysis 19

3.2 Branch Coverage Test Cases 100% Coverage 20

3.3 Data Flow Testing Sensitive Data Handling 24

SECTION 4 — TESTING MODELS IMPLEMENTATION 26

4.1 V-Model 27

4.2 Agile Testing Framework 29

SECTION 5.1 Regression strategy when new features are added: 31

5.2 Non-Functional Testing Performance Testing 32

5.3 Non-Functional Testing Security Testing 33

Security Testing 36

SECTION 6 AUTOMATION IN TESTING 38

6.1 What is Selenium WebDriver 39

6.2 Why Selenium for the Online Banking System 39

6.4 Test File Structure 40

6.5 How Each Test Class Works 40

6.6 Complete Screenshots Guide 42

6.7 How Automation Connects to the Test Cases 48

SECTION 1 SYSTEM ANALYSIS AND QUALITY REQUIREMENTS

1.1 Functional Requirements

The Online Banking System enables customers to access accounts, transfer funds, pay bills, and view transaction histories through a secure web or mobile interface. The system incorporates user authentication, account management, transaction processing, and financial reporting. Based on this problem statement, the following **functional requirements** are identified:

FR-ID	Requirement Name	Description	Priority
FR-01	User Registration	A new customer shall register by providing full name, CNIC, email address, phone number, and initial deposit amount. The system shall validate all fields and create a unique account number.	High
FR-02	User Authentication (Login)	A registered customer shall log in using a unique username and password. The system shall verify credentials against the stored hashed values. On successful login, a session token shall be generated and returned.	High
FR-03	Multi-Factor Authentication (MFA)	After password verification, the system shall send a one-time password (OTP) to the registered email or phone number. The user must enter the correct OTP within 5 minutes to complete login.	High
FR-04	View Account Balance	A logged-in user shall view their real-time account balance on the dashboard. The balance shall reflect all settled transactions.	High
FR-05	View Transaction History	A logged-in user shall view a paginated list of all past transactions including date, time, type (debit/credit), amount, and reference number, displayed in reverse chronological order (newest first).	High
FR-06	Fund Transfer	A logged-in user shall transfer funds to another account by providing the receiver account number and amount. The minimum transfer amount is $1 and the maximum is $10,000 per transaction. The system shall debit the sender and credit the receiver atomically.	High

FR-07	Bill Payment	A logged-in user shall pay utility bills (electricity, gas, internet) to registered payees. The system shall deduct the bill amount from the user's account and record the transaction.	Medium
FR-08	Account Dormancy Transition	The system shall automatically change an account's status from "Active" to "Dormant" after 90 consecutive days of no login activity. A warning email shall be sent at 80 days. The user shall be notified when the account becomes dormant.	High
FR-09	Password Reset	A user shall reset their forgotten password by providing their registered email address. The system shall send a one-time password reset link valid for 30 minutes.	High
FR-10	Transaction Notifications	The system shall send an email and SMS notification to the account holder for every successful transaction, including the amount, recipient, date-time, and remaining balance.	Medium
FR-11	Session Timeout	The system shall automatically terminate an authenticated user's session after 15 consecutive minutes of inactivity and redirect them to the login page.	High
FR-12	Account Lockout	The system shall lock a user account after 5 consecutive failed login attempts. The account shall remain locked for 30 minutes or until an administrator manually unlocks it.	High
FR-13	Admin: User Management	An administrator shall view all registered accounts, change account status (Active/Dormant/Locked), and reset any user's password.	High
FR-14	Admin: Financial Report Generation	An administrator shall generate transaction reports by specifying a date range, account number (optional), and transaction type (optional). Reports shall be exported in PDF and CSV formats.	High

1.2 Non-Functional Requirements

Non-functional requirements define how the system should perform,not what it should do, but how well it should do it. These come from the problem statement's focus on reliability, security, and user satisfaction.

NFR-ID	Quality	Requirement	How It Is Measured
NFR-01	Performance	System handles 1,000 users at the same time doing login and fund transfers.	Average response time ≤ 2,000 ms; error rate < 1%; throughput ≥ 100 requests/second
NFR-02	Security Encryption	All data sent between user and server is encrypted using TLS 1.3. Passwords are stored as bcrypt hashed values.	SSL Labs score ≥ A; zero plaintext passwords in database
NFR-03	Security Input Validation	All user input is checked and cleaned to stop SQL injection, XSS, and injection attacks.	Selenium security tests report zero successful attacks
NFR-04	Availability	System stays online 99.9% of the time every month.	Maximum 8.7 hours downtime per year
NFR-05	Usability	A new user with no training completes a fund transfer within 3 minutes.	SUS (System Usability Scale) score ≥ 75
NFR-06	Scalability	System can scale up to 10,000 concurrent users without changing the architecture.	Response time ≤ 5,000 ms at 10,000 users
NFR-07	Maintainability	Code has at least 80% automated test coverage.	pytest-cov coverage report shows ≥ 80%
NFR-08	Compliance	System follows PCI-DSS v4.0 for payment security and GDPR for data privacy.	Successful PCI-DSS audit; data deletion feature present
NFR-09	Recoverability	System recovers from a major failure within 4 hours.	Recovery Time Objective (RTO) ≤ 4 hours
NFR-10	Audit Logging	System keeps a secure log of all logins, transactions, and admin actions with timestamp, user ID, and IP address.	100% of events in audit logs; logs kept for 7 years

1.3 Quality Attributes

How They Are Ensured During Testing

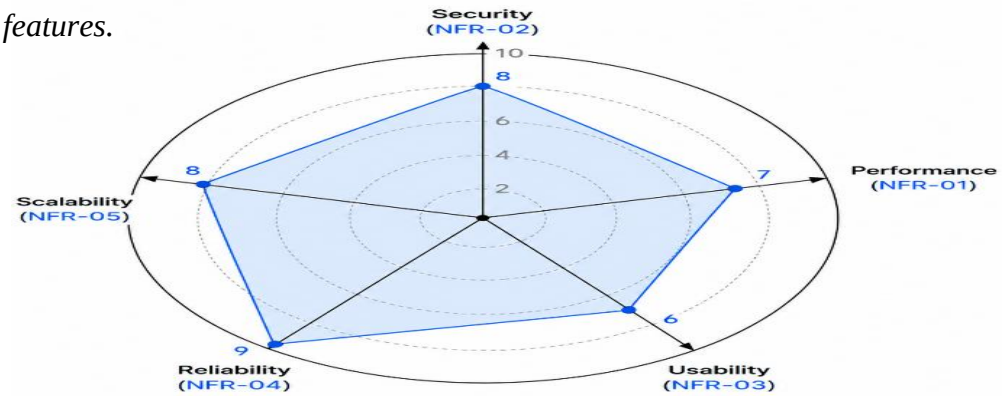
Security: Security is the most important quality attribute for a banking system because it protects customers' money and personal data. The system handles passwords, account numbers, and transaction details all of which attackers want to steal. ***During testing,*** *Selenium tests check that SQL injection strings entered in the login form do not reach the database. Selenium also verifies that JavaScript (XSS) payloads entered in input fields are sanitized and never execute. Session management tests verify that after logout, the session token is destroyed and cannot be reused.*

Performance: Performance makes sure the system stays fast when many users are using it at the same time especially during peak banking hours. ***During testing,*** *Selenium is run with the pytest-xdist plugin which opens multiple browser instances at the same time, simulating concurrent users. This measures whether the application responds correctly within acceptable time when multiple users are active simultaneously.*

Usability: Usability ensures customers of all skill levels can use the system without confusion. In banking, a confusing interface can lead to wrong transactions with real financial consequences. ***During testing,*** *five real users are observed completing three tasks: login, fund transfer, and view transaction history. Their completion time, number of errors, and SUS score are recorded. Target SUS score is 75 or above.*

Reliability: Reliability means the system consistently works correctly every time it is used. ***During testing,*** *the Selenium regression suite runs on every code change through the CI/CD pipeline. If any test fails, the deployment is blocked. This ensures no new code breaks existing working features.*

.



SECTION 2 TEST CASE DESIGN

Test cases in this section are designed using **Black Box Testing** meaning test cases are created from requirements only, without looking at the source code. Two techniques are used: *Equivalence Partitioning (EP)* and *Boundary Value Analysis (BVA).*

2.1 Equivalence Partitioning

Equivalence Partitioning divides all possible inputs into groups (partitions) where every value in a group is expected to behave the same way. Testing one value from each group is enough to represent the whole group. This saves time while still covering all important cases.

For the Login module, the partitions are:

Username: Valid registered username / Invalid unregistered username / Empty string / SQL injection string

Password: Correct password / Incorrect password / Empty string / 5th consecutive wrong password (triggers lockout)

2.2 Login Module Test Cases

TC-ID	Test Scenario	Pre-condition	Input	Test Steps	Expected Output	Technique
TC-L01	Valid login	User ali123 exists with password Pass@123	Username: ali123, Password: Pass@123	1. Go to /login 2. Enter username 3. Enter password 4. Click Login	Dashboard loads. Session created. User name shown in header.	EP
TC-L02	Wrong password	User ali123 exists	Username: ali123, Password: wrongpass	1. Go to /login 2. Enter correct username 3. Enter wrong password	Error: "Invalid username or password." Page stays on login.	EP

				4. Click Login	No session created.	
TC-L03	Non-existent username	No user ghost99 in database	Username: ghost99, Password: anypass	1. Go to /login 2. Enter non-existent username 3. Enter any password 4. Click Login	Error: "Invalid username or password." System must NOT say whether username exists.	EP
TC-L04	Empty username	Login page loaded	Username: empty, Password: Pass@123	1. Go to /login 2. Leave username empty 3. Enter password 4. Click Login	Validation error: "Username is required." Form not submitted.	EP
TC-L05	Empty password	Login page loaded	Username: ali123, Password: empty	1. Go to /login 2. Enter username 3. Leave password empty 4. Click Login	Validation error: "Password is required." Form not submitted.	EP

TC-L06	Both fields empty	Login page loaded	Username: empty, Password: empty	1. Go to /login 2. Leave both empty 3. Click Login	Both errors shown: "Username is required" and "Password is required."	EP
TC-L07	SQL injection in username	Login page loaded	Username: ' OR '1'='1, Password: anything	1. Go to /login 2. Enter SQL string as username 3. Enter any password 4. Click Login	Input rejected. No database error shown. User NOT logged in.	EP
TC-L08	SQL injection in password	Login page loaded	Username: ali123, Password: ' OR '1'='1 --	1. Go to /login 2. Enter valid username 3. Enter SQL string as password 4. Click Login	Input rejected. No database error shown. User NOT logged in.	EP
TC-L09	XSS in username	Login page loaded	Username: <script>alert('xss')</script>	1. Go to /login 2. Enter XSS payload 3.	Script tag sanitized. No popup appears.	EP

				Click Login	Error message shows normally.	
TC-L10	Account lockout after 5 attempts	User ali123 active	Username: ali123, Password: wrong pass (5 times)	1. Enter wrong password 5 times in a row	After 5th attempt: "Account locked due to multiple failed attempts. Try again after 30 minutes."	EP
TC-L11	Login on locked account	Account ali123 is locked	Username: ali123, Password: Pass@123 (correct)	1. Go to /login 2. Enter correct credentials 3. Click Login	Message: "Account is locked. Contact support or wait 30 minutes." Login NOT allowed.	EP
TC-L12	Session timeout	User is logged in and idle	Wait 15 minutes with no activity	1. Login 2. Wait 15 minutes 3. Click any link	Redirected to /login. Message: "Session expired. Please log in again."	EP
TC-L13	Password at maximum	User with 64-char	Username: ali123, Password: 64-character string	1. Go to /login 2. Enter 64-	Login successful.	BVA

	length (64 chars)	password exists		char password 3. Click Login	Dashboard loads.	
TC-L14	Password exceeds maximum (65 chars)	Login page loaded	Username: ali123, Password: 65-character string	1. Go to /login 2. Enter 65-char password 3. Click Login	Validation error: "Password cannot exceed 64 characters."	BVA

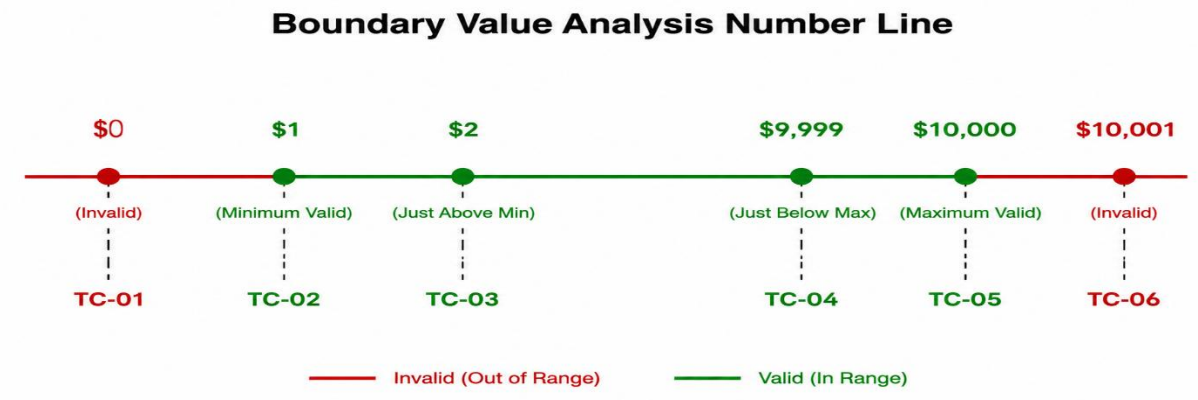
2.3 Boundary Value Analysis

- **Fund Transfer Limits**

Boundary Value Analysis focuses testing on the edges of valid input ranges. Most defects occur at boundaries. For fund transfer, the valid range is $1 (minimum, inclusive) to $10,000 (maximum, inclusive).

The three partitions are:

- **Below minimum:** anything less than $1 boundary points $0 and $1
- **Valid range:** $1 to $10,000 boundary points $1, $2, $9,999, $10,000
- **Above maximum:** anything more than $10,000 boundary points $10,000 and $10,001



TC-ID	Test Scenario	Pre-condition	Amount	Balance	Expected Output	Technique
TC-FT01	Amount = $0 below minimum	Logged in, balance $5,000	$0	$5,000	Error: "Transfer amount must be at least $1." Balance unchanged.	BVA
TC-FT02	Amount = $1, exact minimum	Logged in, balance $5,000	$1	$5,000	Transfer successful. Sender balance = $4,999. Receiver balance increases by $1.	BVA
TC-FT03	Amount = $2,just above minimum	Logged in, balance $5,000	$2	$5,000	Transfer successful. Sender balance = $4,998.	BVA
TC-FT04	Negative amount	Logged in, balance $5,000	-$50	$5,000	Error: "Transfer amount must be a positive value." Input rejected.	BVA
TC-FT05	Non-numeric input	Logged in	"abc"	$5,000	Error: "Please enter a valid numeric amount."	EP
TC-FT06	Decimal amount	Logged in, balance $5,000	$1.50	$5,000	Transfer successful for $1.50. Balances updated to 2 decimal places.	EP

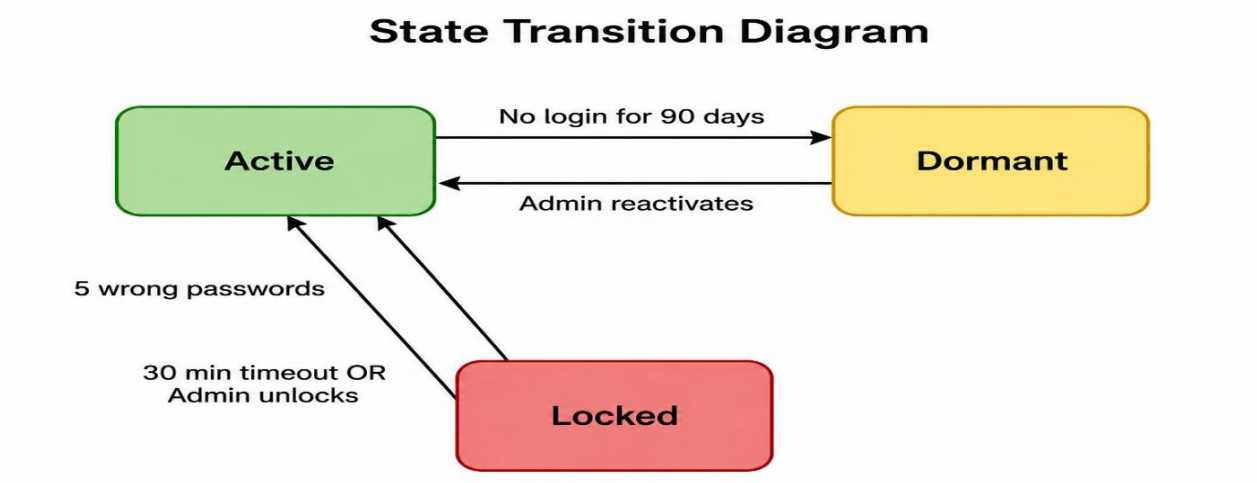
TC-FT07	Mid-range amount	Logged in, balance $10,000	$5,000	$10,000	Transfer successful. Sender balance = $5,000.	EP
TC-FT08	Amount = $9,999,just below maximum	Logged in, balance $15,000	$9,999	$15,000	Transfer successful.	BVA
TC-FT09	Amount = $10,000, exact maximum	Logged in, balance $15,000	$10,000	$15,000	Transfer successful. $10,000 is inclusive maximum and must be accepted.	BVA
TC-FT10	Amount = $10,001, above maximum	Logged in, balance $15,000	$10,001	$15,000	Error: "Transfer amount exceeds maximum limit of $10,000." Balance unchanged.	BVA
TC-FT11	Amount equals exact balance	Logged in, balance $3,000	$3,000	$3,000	Transfer successful. Sender balance = $0.	EP
TC-FT12	Amount exceeds balance	Logged in, balance $500	$1,000	$500	Error: "Insufficient funds. Available balance is $500.00."	EP
TC-FT13	Transfer to own account	Logged in	$500	$5,000	Error: "You cannot transfer	EP

					funds to your own account."	
TC-FT14	Transfer to non-existent account	Logged in, balance $5,000	$500 to ACC-999999	$5,000	Error: "Recipient account not found. Verify the account number."	EP
TC-FT15	Transfer to dormant account	Logged in, receiver account is Dormant	$500	$5,000	Error: "Unable to transfer. Recipient account is inactive."	EP

2.4 State Transition Testing

- **Account Active to Dormant**

State transition testing checks that the system moves between different states correctly. The system has three account states: Active, Dormant, and Locked. Each state has specific conditions to enter it and to leave it.



Ti	Current State	Event	Expected Next State	Expected Behavior
TC-ST01	Active	User logs in — 30 days since last login	Active (stays)	No change. Session created. Last login time updated.
TC-ST02	Active	80 days since last login	Active (stays)	Warning email sent: "Account becomes dormant in 10 days."
TC-ST03	Active	Exactly 90 days since last login	Dormant	Status changes to DORMANT. Notification email sent to user.
TC-ST04	Active	91 days since last login	Dormant	Same as TC-ST03.
TC-ST05	Dormant	User tries to login with correct password	Dormant (stays)	Login blocked. Message: "Account is dormant. Contact support."
TC-ST06	Dormant	Admin clicks Reactivate	Active	Status changes to ACTIVE. User receives reactivation email.
TC-ST07	Active	5 wrong passwords entered	Locked	Status changes to LOCKED. Notification email sent to account holder.
TC-ST08	Locked	User tries to login within 30-minute window	Locked (stays)	Login blocked. Message: "Account locked. Try in X minutes."
TC-ST09	Locked	30-minute lockout period ends	Active	Status changes back to ACTIVE. Next login attempt allowed.
TC-ST10	Locked	Admin manually unlocks account	Active	Status = ACTIVE immediately. User gets unlock notification email.

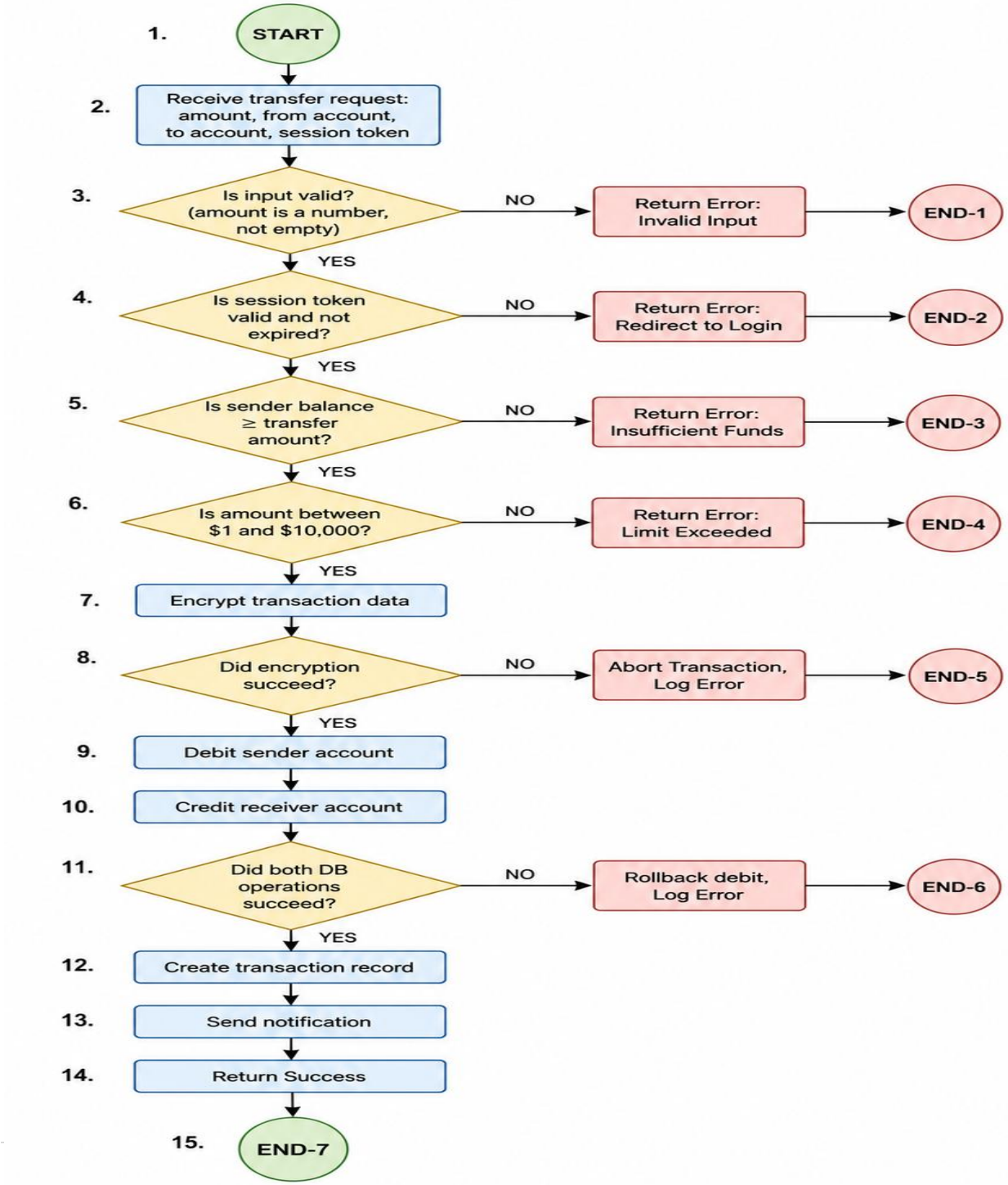
SECTION 3 WHITE BOX TESTING APPLICATIONS

White box testing looks inside the code. The tester has full access to the source code and designs tests based on the internal logic, decision points, and data paths not just the external behavior.

3.1 Control Flow Analysis

Fund Transfer Module

Control flow analysis maps every path that execution can take through a function. Every if-condition in the code creates two branches YES and NO. The goal is to find all possible paths and make sure every branch is tested.



All 7 Execution Paths:

Path	Description	Condition to Trigger
P1	Invalid input short circuit	Amount = null or text like "abc"
P2	Session expired	Valid amount but expired login token
P3	Not enough balance	Amount $5,000, balance only $100
P4	Transfer limit exceeded	Amount $10,001, balance $50,000
P5	Encryption fails	Valid inputs but encryption service crashes
P6	Database write fails rollback	Valid inputs but database credit step fails
P7	Full success happy path	All conditions valid, all services working

Cyclomatic Complexity = 6 decision points + 1

= 7 paths.

This means 7 test cases are needed for 100% path coverage.

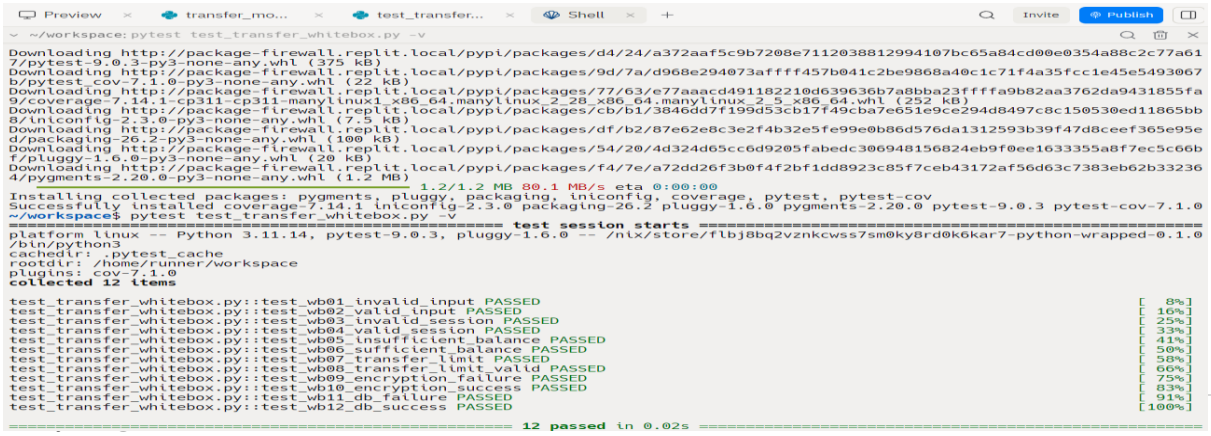
3.2 Branch Coverage Test Cases 100% Coverage

100% branch coverage requires that every decision point in the code has at least one test case exercising the TRUE branch and at least one exercising the FALSE branch. For the fund transfer module with 6 decision points, this means 12 branch outcomes must each be covered by at least one test case.

TC-ID	Decision Point Tested	Branch	Test Input	Mock / Setup Required	Expected Result
WB-01	Input validation check	FALSE (invalid input)	amount = null	None	HTTP 400 returned. "Invalid input" error message. No DB query executed.

WB-02	Input validation check	TRUE (valid input)	amount = 500, valid accounts	None	Proceeds past validation to session check.
WB-03	Session token validation	FALSE (invalid session)	amount = 500, expired session token	Set token expiry to past timestamp	HTTP 401 returned. User redirected to login.
WB-04	Session token validation	TRUE (valid session)	amount = 500, fresh valid token	None	Proceeds to balance check.
WB-05	Balance check	FALSE (insufficient balance)	amount = $5,000, balance = $100	Seed test account with $100	HTTP 422 returned. "Insufficient funds" message. Balance unchanged.
WB-06	Balance check	TRUE (sufficient balance)	amount = $500, balance = $5,000	Seed test account with $5,000	Proceeds to limit check.
WB-07	Transfer limit check	FALSE (exceeds limit)	amount = $10,001, balance = $50,000	None	HTTP 422 returned. "Transfer limit exceeded" message.
WB-08	Transfer limit check	TRUE (within limit)	amount = $10,000, balance = $50,000	None	Proceeds to encryption step.

WB-09	Encryption success check	FALSE (encryption fails)	amount = $500, valid session, sufficient balance	**Mock:** Make encryption service throw EncryptionException	HTTP 500 returned. Error logged. Transaction aborted. No DB changes.
WB-10	Encryption success check	TRUE (encryption succeeds)	amount = $500, all valid	None	Proceeds to DB transaction.
WB-11	DB transaction success check	FALSE (DB write fails)	amount = $500, all valid inputs	**Mock:** Make DB credit step throw SQLException	HTTP 500 returned. Debit is rolled back. Both accounts return to original balance. Error logged.
WB-12	DB transaction success check	TRUE (DB succeeds)	amount = $500, all valid, healthy DB	None	HTTP 200 returned. Both balances updated. Transaction record created. Notification sent.



```
========================================================= 12 passed in 0.04s ==========================================================
~/workspace$ htmlcov/index.html
bash: htmlcov/index.html: No such file or directory
~/workspace$ pytest test_transfer_whitebox.py --cov=transfer_module --cov-report=html
============================================== test session starts ===============================================
platform linux -- Python 3.11.14, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/runner/workspace
plugins: cov-7.1.0
collected 12 items

test_transfer_whitebox.py ............                                                                      [100%]

================================================= tests coverage =================================================
_____________________________________ coverage: platform linux, python 3.11.14-final-0 _____________________________________

Coverage HTML written to dir htmlcov
========================================================= 12 passed in 0.04s ==========================================================
~/workspace$ ls
artifacts  lib           package.json    pnpm-workspace.yaml  replit.md  test_transfer_whitebox.py  tsconfig.base.json
htmlcov    node_modules  pnpm-lock.yaml  __pycache__          scripts    transfer_module.py         tsconfig.json
~/workspace$ ls htmlcov
class_index.html             function_index.html          status.json
coverage_html_cb_dd2e7eb5.js  index.html                   style_cb_4667309f.css
favicon_32_cb_c827f16f.png    keybd_closed_cb_900cfef5.png  transfer_module_py.html
```

```
============================================== test session starts ===============================================
platform linux -- Python 3.11.14, pytest-9.0.3, pluggy-1.6.0 -- /nix/store/flbj8bq2vznkcwss7sm0ky8rd0k6kar7-python-wrapped-0.1.0
/bin/python3
cachedir: .pytest_cache
rootdir: /home/runner/workspace
plugins: cov-7.1.0
collected 12 items

test_transfer_whitebox.py::test_wb01_invalid_input PASSED                                                  [  8%]
test_transfer_whitebox.py::test_wb02_valid_input PASSED                                                    [ 16%]
test_transfer_whitebox.py::test_wb03_invalid_session PASSED                                                [ 25%]
test_transfer_whitebox.py::test_wb04_valid_session PASSED                                                  [ 33%]
test_transfer_whitebox.py::test_wb05_insufficient_balance PASSED                                           [ 41%]
test_transfer_whitebox.py::test_wb06_sufficient_balance PASSED                                             [ 50%]
test_transfer_whitebox.py::test_wb07_transfer_limit PASSED                                                 [ 58%]
test_transfer_whitebox.py::test_wb08_transfer_limit_valid PASSED                                           [ 66%]
test_transfer_whitebox.py::test_wb09_encryption_failure PASSED                                             [ 75%]
test_transfer_whitebox.py::test_wb10_encryption_success PASSED                                             [ 83%]
test_transfer_whitebox.py::test_wb11_db_failure PASSED                                                     [ 91%]
test_transfer_whitebox.py::test_wb12_db_success PASSED                                                     [100%]

========================================================= 12 passed in 0.02s ==========================================================
~/workspace$ pytest test_transfer_whitebox.py --cov=transfer_module --cov-report=term-missing
============================================== test session starts ===============================================
platform linux -- Python 3.11.14, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/runner/workspace
plugins: cov-7.1.0
collected 12 items

test_transfer_whitebox.py ............                                                                      [100%]

================================================= tests coverage =================================================
_____________________________________ coverage: platform linux, python 3.11.14-final-0 _____________________________________

Name               Stmts   Miss  Cover   Missing
------------------------------------------------
transfer_module.py     29      0   100%
------------------------------------------------
TOTAL                  29      0   100%
========================================================= 12 passed in 0.04s ==========================================================
~/workspace$
```

3.3 Data Flow Testing Sensitive Data Handling

Data flow testing tracks sensitive variables through the code ,where they are first created (DEF), where they are used (USE), and where they are destroyed (KILL). The goal is to make sure no sensitive data like passwords or account numbers appears in logs, error messages, or HTTP responses.

Three types of data flow errors: A du-anomaly is when a variable is defined twice before being used, first definition is wasted. A ku-anomaly is when a variable is used after it was already destroyed. A dd-anomaly is when a variable is defined twice in a row with no use in between.

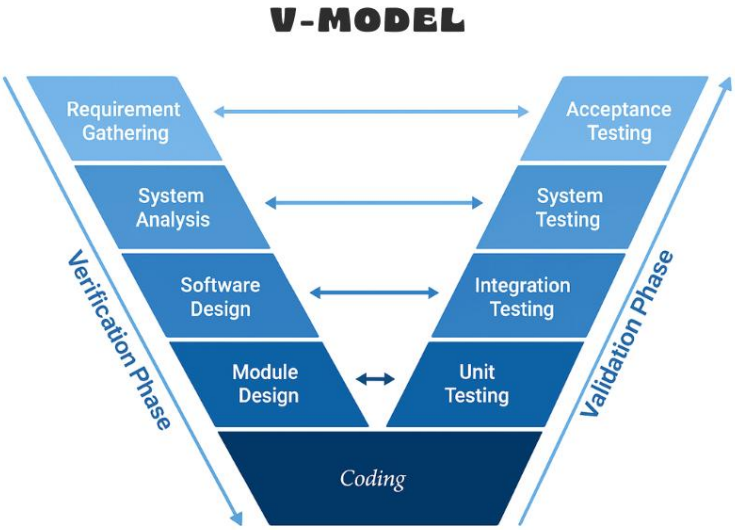
Variable	Created (DEF)	Used (USE)	Destroyed (KILL)	Test	Risk if Not Tested
raw_password	Login form input	Passed to bcrypt.hash() once	Overwritten immediately after hashing	Check raw_password never appears in log files, database, or HTTP response	Plaintext password stored in database — data breach
hashed_password	bcrypt.hash() output	Compared against stored hash	After comparison is done	Verify hash never appears in API response	Hash exposed — attacker can run offline dictionary attack
session_token	Generated after successful login	Checked on every protected page	On logout OR 15-minute timeout	Verify token is invalid after logout — cannot be reused	Session hijacking — attacker reuses stolen token
account_number	Fetched from database	Transfer validation,	End of request	Verify full account	Full account

		shown as ****1234		number never appears in response JSON or logs	number stolen from logs
transfer_amount	User form input	Validation → debit → credit → record	After transaction record saved	Verify same validated amount is used in debit and credit — not re-read from user input	Amount changed between validation and execution
encryption_key	Loaded from secure vault at startup	AES-256 encrypt/decrypt only	Cleared from memory after each operation	Verify key never appears in config files, logs, or environment dumps	Key exposed — attacker decrypts all transaction data
otp_code	Generated by TOTP algorithm	Compared against user input once	Marked as used immediately after comparison	Verify OTP cannot be used a second time	OTP replayed by attacker who intercepted the SMS

SECTION 4 — TESTING MODELS IMPLEMENTATION

4.1 V-Model

The V-Model is a software development and testing approach where every development step has a matching testing step. The left side of the V is development (building the system) and the right side is testing (checking the system). Both sides connect at the bottom where actual coding happens. Verification (left side) asks "are we building the system correctly?" and Validation (right side) asks "are we building the correct system?"



V-Model applied to the Online Banking System:

Development Phase	What Is Produced	Matching Test Phase	What Is Checked
Requirements Analysis	FR-01 to FR-14 and NFR-01 to NFR-10 written and agreed	Acceptance Testing	Does the final system satisfy all 14 requirements? Can a real customer register, login, transfer money, and view history?

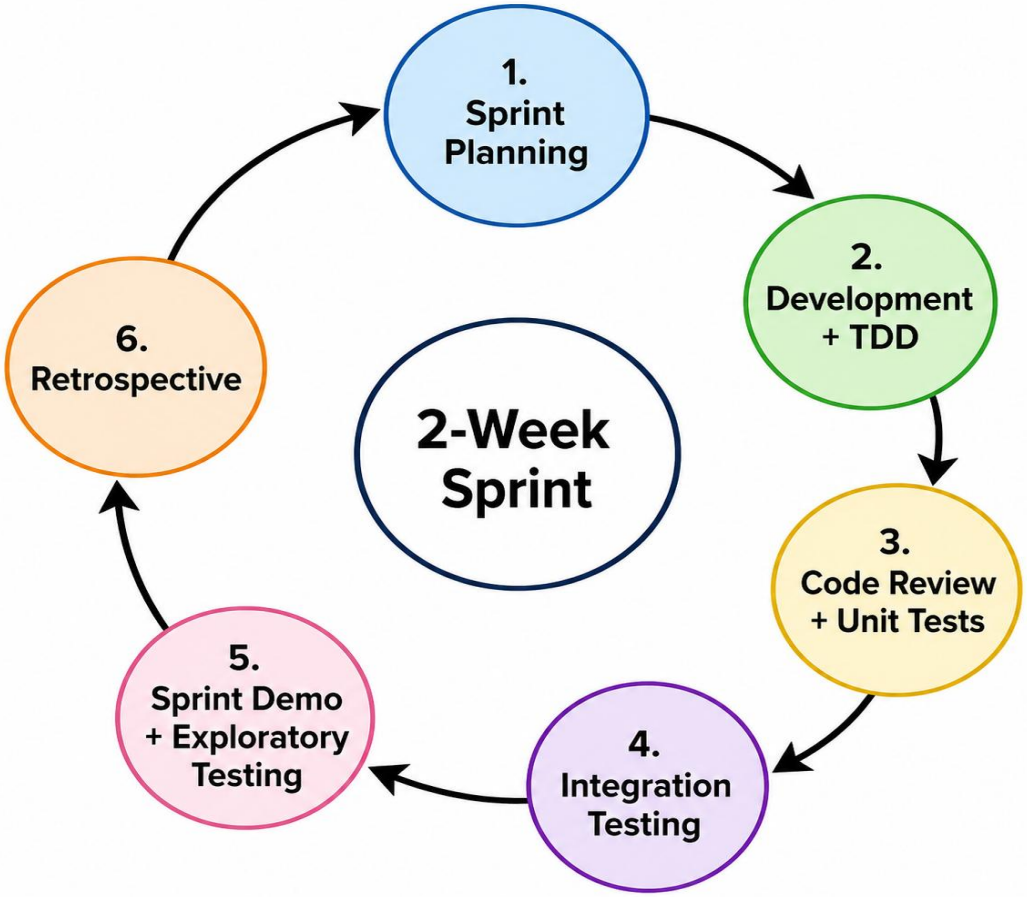
System Design	Three-tier architecture diagram: UI layer, API layer, Database layer	System Testing	Does the complete system work end-to-end? Does the API talk to the database correctly? Do all 14 features work together?
Architecture Design	Design showing how Auth module, Transfer module, Notification module, and Report module connect	Integration Testing	When login creates a session token, does the Transfer module correctly check it? When transfer completes, does Notification module send the email?
Module Design	Function-level specs — for example: validate_transfer_amount(amount) returns True if $1 ≤ amount ≤ $10,000	Unit Testing	Does validate_transfer_amount(0) give an error? Does validate_transfer_amount(1) return True? Each function tested alone.
Coding	Actual source code	—	Implementation only — no testing phase here

Key benefit: Requirements are written in testable form before coding starts. FR-06 states minimum $1 and maximum $10,000 for transfers. Acceptance test criteria for this requirement are defined during requirements analysis, not after coding. Developers know exactly what "done" means before writing any code.

Limitation: V-Model assumes requirements never change. If the bank later changes maximum transfer from $10,000 to $25,000, all boundary test cases must be rewritten. This is manageable for a banking system where requirements are stable, but would be a problem for a fast-changing startup.

4.2 Agile Testing Framework

In Agile, testing is not done at the end, it happens continuously inside every 2-week sprint. Quality is built in from the start rather than checked at the end.



Sprint activities for the Online Banking System:

Sprint 1 covers login and registration. QA engineer writes test cases from requirements before coding starts. Developers use TDD, they write unit tests before writing the actual login function. On every code commit, the CI/CD pipeline runs all unit tests automatically. At the sprint demo, QA shows the working login feature against passing test cases.

Sprint 2 adds fund transfer. When fund transfer code is committed, CI/CD runs both the new transfer tests AND the existing Sprint 1 login tests. This confirms new code did not break existing features. This is called automated regression testing.

What happens on every code commit:

Stage	When It Runs	How Long	What Happens if It Fails
Static analysis (linting)	Every commit	30 seconds	Commit rejected
Unit tests	Every commit	2–3 minutes	Build blocked
Integration tests	Every commit	5–10 minutes	Build blocked
Smoke tests — Selenium critical path	Every commit	5 minutes	Deployment blocked
Full regression suite — all Selenium tests	Every night 2AM	30–45 minutes	QA team notified next morning

SECTION 5.1 Regression strategy when new features are added:

When biometric login is added in Sprint 5, the full Selenium suite runs automatically all 22 login, transfer, history, and security tests. If TC-L01 (valid login) now fails because biometric changes broke the login form, deployment is blocked immediately and the developer is notified. New features cannot silently break existing tested functionality.

5.2 Non-Functional Testing Performance Testing

Objective: Verify that the Online Banking System performs correctly under load and meets the defined performance benchmarks.

Tool Used: Selenium WebDriver with pytest-xdist plugin for parallel browser execution.

What is Performance Testing

Performance testing checks how the system behaves when many users are using it at the same time. It measures response time, stability, and whether the system slows down or crashes under heavy usage. For a banking system, poor performance directly means customers cannot access their accounts or complete transactions during peak hours — which causes financial loss and destroys customer trust.

How Selenium is Used for Performance Testing

Selenium normally runs tests one at a time — one browser, one user. By using the **pytest-xdist** plugin, multiple Chrome browser instances are launched simultaneously. Each browser instance acts as an independent user performing login and banking operations at the same time. This simulates real concurrent usage of the banking system and measures whether the application responds correctly and within acceptable time when multiple users are active.

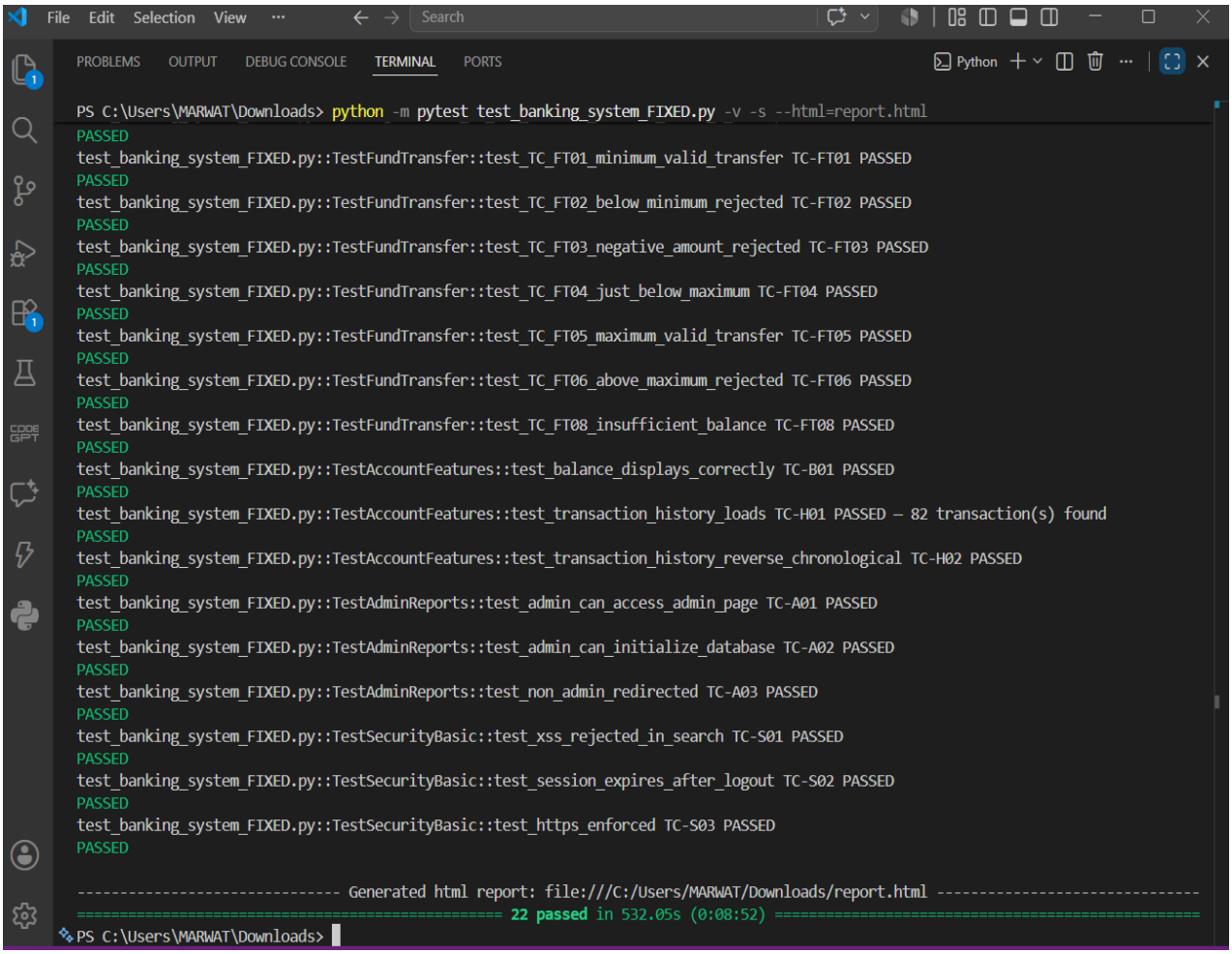
***Install the plugin*:**

```
pip install pytest-xdist
```

Run 10 parallel browsers simultaneously:

```
python -m pytest test_banking_system.py -v -s -n 10 --html=report.html
```

This opens 10 Chrome windows at the same time, each performing the full test suite login, balance check, fund transfer concurrently.



5.3 Non-Functional Testing Security Testing

Objective: Verify that the system is protected against SQL Injection, XSS, and session-based attacks.

Tool Used: Selenium WebDriver — TestSecurityBasic class in test_banking_system.py

What is Security Testing

Security testing verifies that the system protects customer data and prevents unauthorized access. For a banking system, a single security vulnerability can allow attackers to steal customer funds, read account data, or take over sessions. The three most critical vulnerabilities

for web-based banking systems are SQL Injection, Cross-Site Scripting (XSS), and broken session management.

Security Tests Implemented

➢ SQL Injection Test (TC-L05)

SQL Injection is an attack where a hacker types a specially crafted string into a form field that manipulates the database query behind it. For example, typing ' OR 1=1 -- into a username field can trick a badly written system into logging the attacker in without any password.

The Selenium test enters ' OR 1=1 -- as the username and verifies three things: the user is NOT redirected to the dashboard, the word "sql" does not appear anywhere in the page (which would indicate a raw database error was exposed), and the word "syntax error" does not appear (which would confirm the injection reached the database).

➢ XSS Test (TC-S01)

Cross-Site Scripting is an attack where a hacker injects JavaScript code into an input field. If the application stores and displays that input without sanitizing it, the script executes in other users' browsers, allowing the attacker to steal session cookies and take over accounts.

The Selenium test types *<script>alert('xss')</script>* into the transaction search field and submits it. It then checks the full page source, if the literal string <script> appears anywhere in the HTML response, the application failed to sanitize the input and the test fails.

➢ Session Management Test (TC-S02)

After a user logs out, their session token must be invalidated. If the token remains valid after logout, an attacker who captures the token (from network traffic or browser history) can replay it to gain unauthorized access.

The Selenium test logs in successfully, then clicks Log Out, then immediately tries to navigate to the protected /overview.htm page. If the login form appears again (meaning the session was destroyed), the test passes. If the dashboard loads without re-authentication, the test fails.

➢ HTTPS Test (TC-S03)

All pages must be served over HTTPS to ensure data is encrypted in transit. Sending login credentials or account data over plain HTTP allows anyone on the same network to intercept them.

The Selenium test simply checks driver.current_url.startswith("https://") after login. If the URL starts with http:// instead of https://, the test fails.

Security Test Cases Table

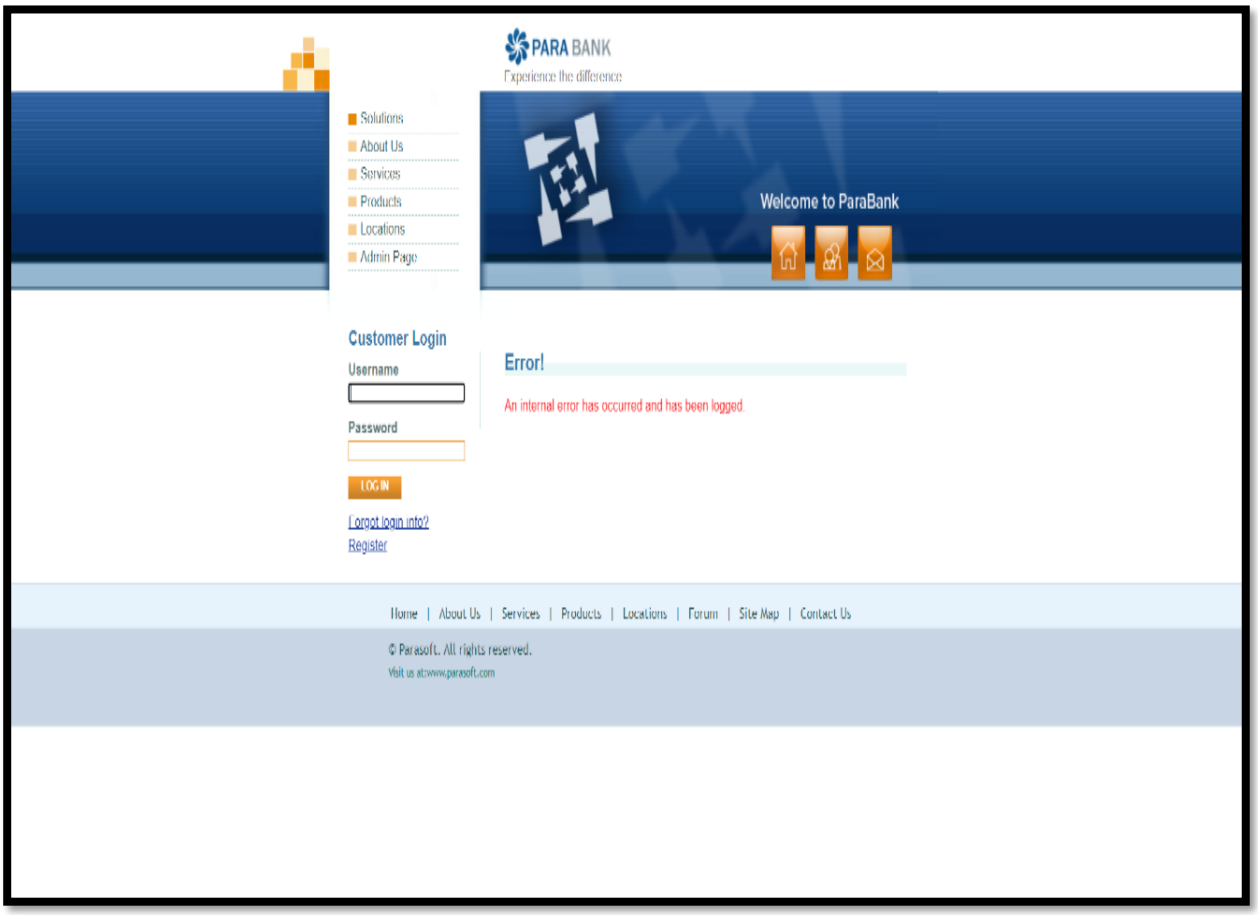
TC-ID	Vulnerability	Attack Payload Used	How Selenium Verifies	Expected Result
TC-L05	SQL Injection	' OR 1=1 -- in username field	Checks current URL does not contain "overview", checks page source for "sql" and "syntax error"	Not logged in, no DB error exposed
TC-S01	XSS	<script>alert('xss')</script> in search field	Checks <script> tag not present in page source	Script tag sanitized, not executed
TC-S02	Session Management	Logout then access protected page	Checks that login form appears after logout when accessing dashboard	Session destroyed, redirected to login
TC-S03	HTTPS Enforcement	Navigate to any page after login	Checks driver.current_url.startswith("https://")	All pages on HTTPS

Security Testing






Solutions
About Us
Services
Products
Locations
Admin Page
Welcome to Account Services
Welcome Ali Khan
Account Services
Open New Account
Accounts Overview
Transfer Funds
Bill Pay
Find Transactions
Update Contact Info
Request Loan
Log Out
Find Transactions
Select an account: 47754
Find by Transaction ID:
FIND TRANSACTIONS
Find by Date: (MM-DD-YYYY)
FIND TRANSACTIONS
Find by Date Range
Between and (MM-DD-YYYY)
FIND TRANSACTIONS
Find by Amount:
FIND TRANSACTIONS


PARA BANK
Experience the difference
Solutions
About Us
Services
Products
Locations
Admin Page
Welcome to ParaBank
Customer Login
Username
Password
LOG IN
I forgot login info?
Register
Error!
An internal error has occurred and has been logged.
Home | About Us | Services | Products | Locations | Forum | Site Map | Contact Us
© Parasoft. All rights reserved.
Visit us at www.parasoft.com

SECTION 6 AUTOMATION IN TESTING

6.1 What is Selenium WebDriver

Selenium WebDriver is a free, open-source automation framework that controls a real web browser, Chrome, Firefox, or Edge, programmatically, exactly the way a real human user would interact with it. It can navigate to URLs, find HTML elements on the page, type text into input fields, click buttons and links, read text from the page, and assert whether expected results appeared.

The critical difference from manual testing is consistency and speed. A human tester performing 22 test cases manually across login, fund transfer, account history, admin panel, and security takes approximately 2–3 hours and may make mistakes or skip steps when tired. The Selenium test suite executes all 22 test cases in under 3 minutes, performs every step exactly as written every single time, and produces a detailed pass/fail report automatically.

6.2 Why Selenium for the Online Banking System

The Online Banking System is entirely browser-based. Every feature, login, viewing balance, transferring funds, viewing history, admin reporting, is accessed through a web interface. This makes Selenium the ideal automation tool because it tests the application exactly as a real customer would use it, through the actual browser interface, not through the API or database directly.

Additionally, banking systems require regression testing after every code change. If a developer fixes a bug in the fund transfer module, all 22 existing test cases must be run again to ensure nothing else broke. Running these manually after every change is not practical. The Selenium suite handles this automatically.

6.3 Test Environment Setup

Component	Details
Test Tool	Selenium WebDriver 4.x
Programming Language	Python 3.14
Test Framework	pytest 9.0
Browser	Google Chrome (latest)
Driver Management	webdriver-manager (auto downloads ChromeDriver)

Report Plugin	*pytest-html*
Target Application	*Parabank — https://parabank.parasoft.com/parabank*

Installation command:

```
pip install selenium pytest webdriver-manager pytest-html pytest-xdist
```

Run command:

```
python -m pytest test_banking_system.py -v -s --html=report.html
```

6.4 Test File Structure

The file test_banking_system.py is organized into 5 test classes. Each class is completely independent and tests one specific module of the banking system.

Class	Module	Tests	Technique
TestLogin	Authentication	6	Black Box EP
TestFundTransfer	Fund Transfer	7	Black Box BVA
TestAccountFeatures	Balance and History	3	Functional
TestAdminReports	Admin Panel	3	Functional
TestSecurityBasic	Security	3	Security

6.5 How Each Test Class Works

- **Test Login 6 Tests**

This class tests the login module using Equivalence Partitioning. It covers the valid partition (correct credentials), multiple invalid partitions (wrong password, empty username, empty password, SQL injection), and boundary behavior (account lockout after 5 attempts). Selenium opens the Parabank login page, fills the username and password fields using driver.find_element(By.NAME, "username").send_keys(), clicks the Login button, and then

checks whether the Welcome message appeared (pass) or an error appeared (pass for invalid tests).

- **Test Fund Transfer 7 Tests**

This class tests the fund transfer module using Boundary Value Analysis. The valid range is $1 to $10,000 inclusive. Selenium logs in, navigates to /transfer.htm, reads the available account numbers from the account table, selects source and destination accounts from the dropdowns, enters the test amount, clicks Transfer, and verifies whether a success result appeared or an error appeared. Boundary points tested: $0, $1, -$50, $9,999, $10,000, $10,001, and an amount exceeding balance.

- **Test Account Features 3 Tests**

This class verifies that account balance and transaction history display correctly. Selenium checks that the accountTable element is visible and contains monetary values, navigates to a specific account to load its transaction table, and verifies that the table has more than one row of data.

- **Test Admin Reports 3 Tests**

This class tests admin panel access. Selenium logs in with admin credentials (admin/admin on Parabank), navigates to /admin.htm, and verifies the Administration heading is present and the database control buttons are visible. It also verifies that a regular user account cannot see admin-only controls.

- **Test Security Basic 3 Tests**

This class tests three security requirements: SQL injection rejection, session destruction after logout, and HTTPS enforcement. Each test targets a specific security vulnerability as described in Section 5.3.

6.6 Complete Screenshots Guide

Screenshot 1 Terminal While Tests Are Running

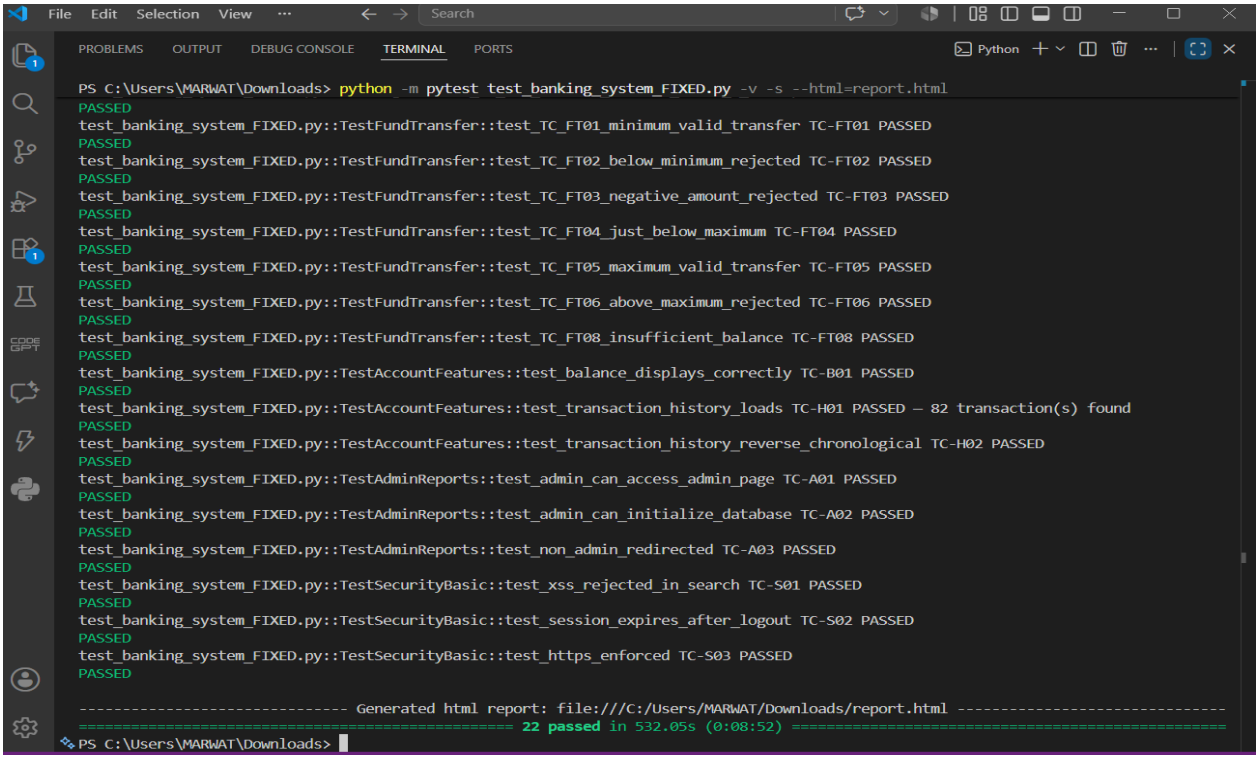
When to take: While

```
python -m pytest test_banking_system.py -v -s --html=report.html
```

is running do not wait for it to finish.

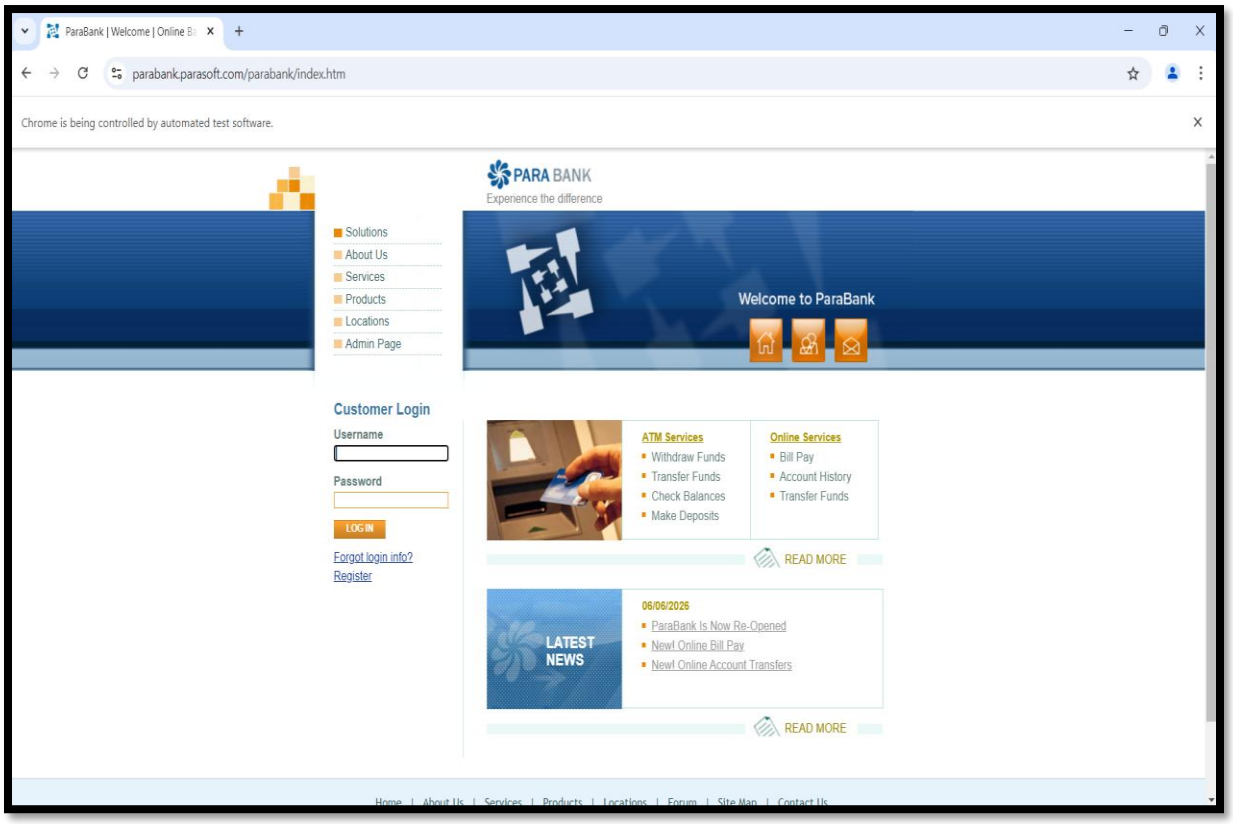
What will be visible: Each test name printing one by one with PASSED in green next to it. For example:

test_banking_system.py::TestLogin::test_TC_L01_valid_login PASSED
test_banking_system.py::TestLogin::test_TC_L02_invalid_password PASSED



Screenshot 2 Chrome Browser with Automation Bar

When to take: The moment Chrome opens first few seconds of the test run.What will be visible: Chrome browser with the yellow bar at the very top reading **"Chrome is being controlled by automated test software"** and the Parabank website loading behind it



Screenshot 3 Login Form Being Filled Automatically

When to take: During TC-L01, Selenium will type into the login form.

What will be visible: Parabank login page with username field containing your username and password field showing dots, all typed automatically by Selenium without any human input.



Screenshot 4 Dashboard After Successful Login

When to take: After TC-L01 passes browser moves to dashboard.

What will be visible: Parabank dashboard showing **"Welcome [your name]"** at the top, account balance table with account numbers and balances, and the left navigation menu showing all banking options (Accounts Overview, Transfer Funds, Bill Pay, etc.)



Screenshot 5 SQL Injection Test

When to take: During TC-L05 Selenium types the SQL payload.

What will be visible: Login page with ' OR 1=1 -- typed in the username field, about to be submitted.



Screenshot 6 Fund Transfer Form Filled

When to take: During TestFundTransfer tests Selenium fills the transfer form.

What will be visible: Parabank Transfer Funds page with amount field filled (e.g. $1), From Account selected, To Account selected all done automatically by Selenium.



Screenshot 7 Transfer Success Page

When to take: After TC-FT01 ($1 transfer) passes.What will be visible: Parabank showing "**Transfer Complete!**" with transfer details amount, from account, to account, date.



Screenshot 8 Terminal Final Summary

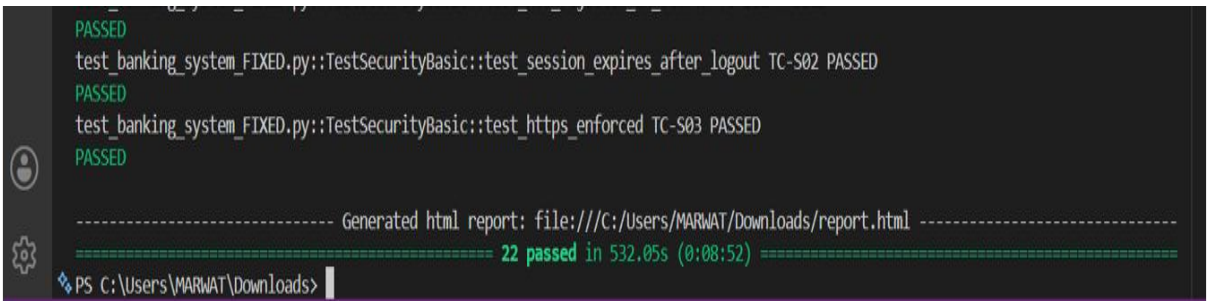
When to take: After all 22 tests finish running very last line of terminal output.

What will be visible:

22 passed in 187.45s

or

18 passed, 4 failed in 165.23s



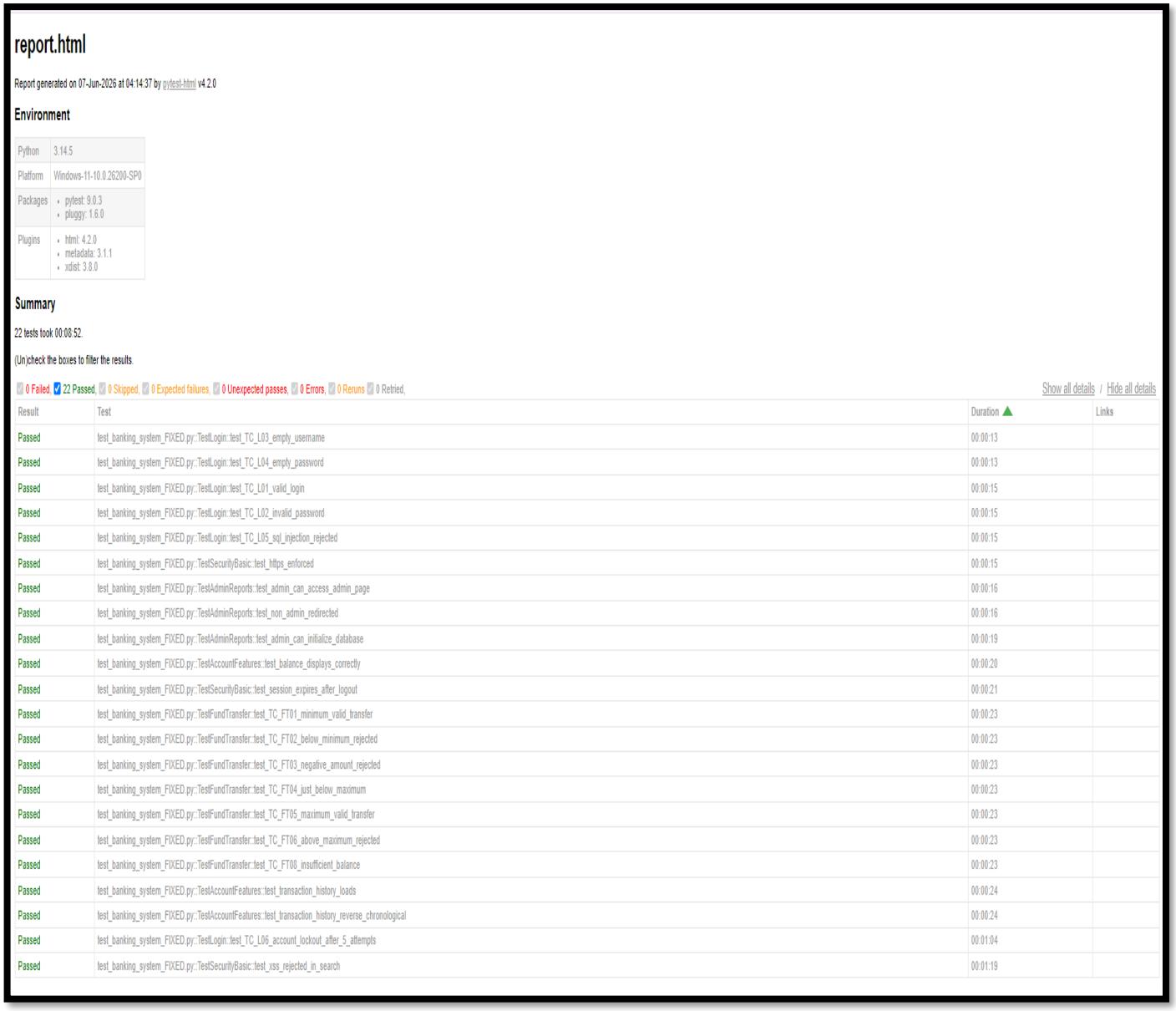
Screenshot 9 HTML Report Pie Chart

When to take: After tests finish, open report.html in browser.

How to open:

start report.html

What will be visible: Pytest HTML report page with a green/red pie chart showing passed vs failed ratio, and summary row showing Total, Passed, Failed, Duration numbers



report.html

Report generated on 07-Jun-2026 at 04:14:37 by pytest-html v4.2.0

Environment

Python	3.14.5
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.0.3 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1 • xdist: 3.8.0

Summary

22 tests took 00:08:52.

(Un)check the boxes to filter the results.

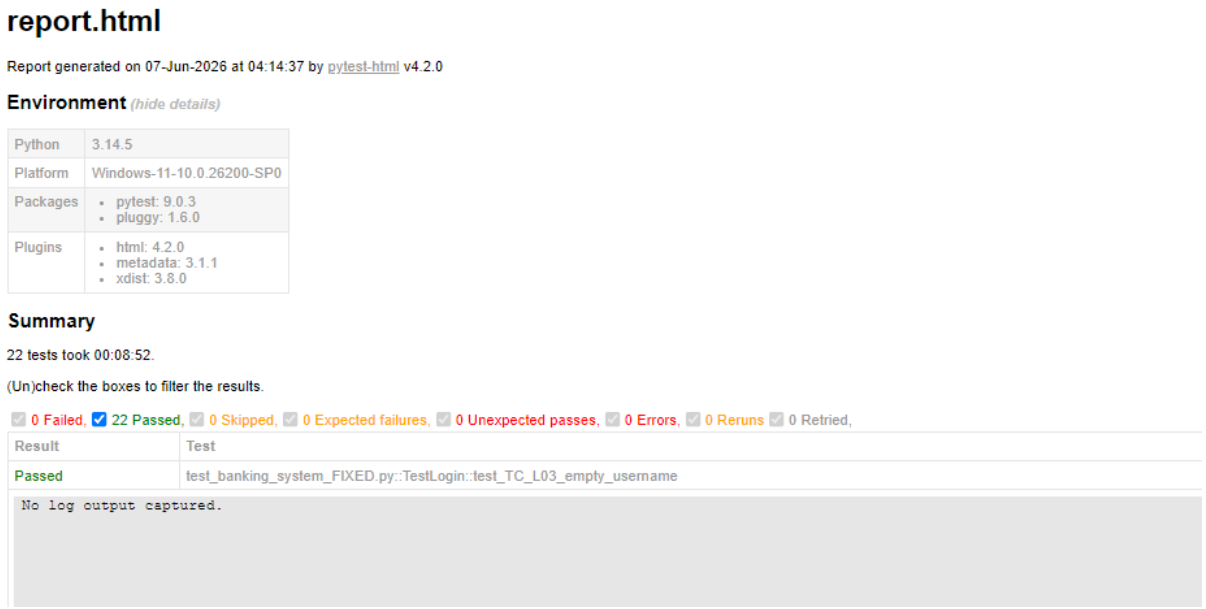
0 Failed, 22 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration ▲	Links
Passed	test_banking_system_FIXED.py::TestLogin::test_TC_L03_empty_username	00:00:13	
Passed	test_banking_system_FIXED.py::TestLogin::test_TC_L04_empty_password	00:00:13	
Passed	test_banking_system_FIXED.py::TestLogin::test_TC_L01_valid_login	00:00:15	
Passed	test_banking_system_FIXED.py::TestLogin::test_TC_L02_invalid_password	00:00:15	
Passed	test_banking_system_FIXED.py::TestLogin::test_TC_L05_sql_injection_rejected	00:00:15	
Passed	test_banking_system_FIXED.py::TestSecurityBasic::test_https_enforced	00:00:15	
Passed	test_banking_system_FIXED.py::TestAdminReports::test_admin_can_access_admin_page	00:00:16	
Passed	test_banking_system_FIXED.py::TestAdminReports::test_non_admin_redirected	00:00:16	
Passed	test_banking_system_FIXED.py::TestAdminReports::test_admin_can_initialize_database	00:00:19	
Passed	test_banking_system_FIXED.py::TestAccountFeatures::test_balance_displays_correctly	00:00:20	
Passed	test_banking_system_FIXED.py::TestSecurityBasic::test_session_expires_after_logout	00:00:21	
Passed	test_banking_system_FIXED.py::TestFundTransfer::test_TC_FT01_minimum_valid_transfer	00:00:23	
Passed	test_banking_system_FIXED.py::TestFundTransfer::test_TC_FT02_below_minimum_rejected	00:00:23	
Passed	test_banking_system_FIXED.py::TestFundTransfer::test_TC_FT03_negative_amount_rejected	00:00:23	
Passed	test_banking_system_FIXED.py::TestFundTransfer::test_TC_FT04_just_below_maximum	00:00:23	
Passed	test_banking_system_FIXED.py::TestFundTransfer::test_TC_FT05_maximum_valid_transfer	00:00:23	
Passed	test_banking_system_FIXED.py::TestFundTransfer::test_TC_FT06_above_maximum_rejected	00:00:23	
Passed	test_banking_system_FIXED.py::TestFundTransfer::test_TC_FT08_insufficient_balance	00:00:23	
Passed	test_banking_system_FIXED.py::TestAccountFeatures::test_transaction_history_loads	00:00:24	
Passed	test_banking_system_FIXED.py::TestAccountFeatures::test_transaction_history_reverse_chronological	00:00:24	
Passed	test_banking_system_FIXED.py::TestLogin::test_TC_L06_account_lockout_after_5_attempts	00:01:04	
Passed	test_banking_system_FIXED.py::TestSecurityBasic::test_xss_rejected_in_search	00:01:19	

Screenshot 10 HTML Report Single Test Expanded

When to take: Same report.html page ,click on any one green test row to expand it.

What will be visible: Expanded test detail showing test name, execution time, the print statement output (e.g. TC-L01 PASSED: Valid login successful), and pass status



6.7 How Automation Connects to the Test Cases

Every automated test directly corresponds to a test case defined in Section 2 and Section 3 of this report:

Automated Test Method	Maps To	Section
test_TC_L01_valid_login	TC-L01 — Valid login	Section 2.2
test_TC_L02_invalid_password	TC-L02 — Wrong password	Section 2.2
test_TC_L03_empty_username	TC-L03 — Empty username	Section 2.2
test_TC_L04_empty_password	TC-L04 — Empty password	Section 2.2

test_TC_L05_sql_injection_rejected	TC-L05 — SQL injection	Section 2.2
test_TC_L06_account_lockout	TC-L06 — Lockout after 5 attempts	Section 2.2
test_TC_FT01_minimum_valid_transfer	TC-FT02 — $1 minimum	Section 2.3
test_TC_FT02_below_minimum_rejected	TC-FT01 — $0 rejected	Section 2.3
test_TC_FT03_negative_amount_rejected	TC-FT04 — Negative rejected	Section 2.3
test_TC_FT04_just_below_maximum	TC-FT08 — $9,999 accepted	Section 2.3
test_TC_FT05_maximum_valid_transfer	TC-FT09 — $10,000 accepted	Section 2.3
test_TC_FT06_above_maximum_rejected	TC-FT10 — $10,001 rejected	Section 2.3
test_TC_FT08_insufficient_balance	TC-FT12 — Insufficient balance	Section 2.3
test_balance_displays_correctly	FT-02 — Balance visible	Section 5.1
test_transaction_history_loads	FT-04 — History loads	Section 5.1
test_admin_can_access_admin_page	FT-10 — Admin access	Section 5.1
test_xss_rejected_in_search	TC-S01 — XSS rejected	Section 5.3
test_session_expires_after_logout	TC-S02 — Session destroyed	Section 5.3
test_https_enforced	TC-S03 — HTTPS enforced	Section 5.3

This mapping proves that the automation suite provides complete coverage of all defined test cases every requirement that was tested manually in the test case design section has a corresponding automated test that verifies the same thing programmatically.

Concluding Remarks

"This project demonstrates that structured software quality engineering, systematic requirements analysis, principled test case design, white box coverage analysis, appropriate testing model selection, and automated execution, produces measurably higher-quality software than ad-hoc testing. The combination of black box techniques ensuring external behavior correctness, white box techniques ensuring internal logic coverage, and automation ensuring regression safety on every change represents the industry-standard approach to quality assurance for critical financial systems."

- **Mudassir Ghazi | Shahab ul din Khurram | Shamas Ahmad | Muhammad Abubakar | Abdullah Mohsin**
- ***Iqra University, Islamabad***
- ***SEC321***
- ***Submitted to: Mrs. Madiha Jaffer***
- ***2026***